

University Assignment Management System

Software Engineering Project Report

Team Members

Jerom Raed, Mina Emile, Daniel Wadie, Rober Ramzy

Course

Software Engineering

Submission Date

13th May 2026

2. Software Development Lifecycle (SDLC)

This project was developed using an Agile software development methodology, specifically a 7-day iterative approach utilizing daily stand-ups and continuous integration. This process allowed for the rapid delivery of a working system while responding to implementation challenges dynamically.

2.1 Requirements Phase

Problem Statement: University departments currently rely on shared Google Sheets for assignment tracking, which presents several critical pain points: anyone can overwrite another person's data, there is no flexibility for per-section deadline extensions, students cannot track their personal completion status, and approaching deadlines lack automated notifications.

System Scope: The Assignment Management System (AMS) is a web application that replaces shared spreadsheets to centralize assignment management across all departments, levels, and courses. The system supports role-based access, per-section deadline overrides, and personal completion trackers. It operates strictly as a tracking and routing tool; students access submission links but do not upload files directly to the platform.

Functional Requirements:

- The system shall allow students to browse the course catalogue and self-enroll in their respective sections and lab groups without requiring administrative approval.
- The system shall enable students to view their assignments, see accurately resolved deadlines (accounting for section overrides), and mark assignments as "done" using a personal checkbox.
- The system shall allow Course Coordinators to create, edit, and delete assignments, as well as set a default system-wide deadline for each assignment.
- The system shall allow Course Coordinators to create section-specific deadline overrides without needing to duplicate the entire assignment.
- The system shall provide an Admin panel to manage the academic hierarchy (Departments, Terms, Courses, Sections) and approve pending Coordinator accounts.

Non-Functional Requirements:

- **Performance:** The system must resolve the correct deadline (checking for section overrides before falling back to defaults) instantly upon a student loading their dashboard.
- **Security:** The system must enforce Role-Based Access Control (RBAC) ensuring coordinators can only modify their assigned courses and cannot access unauthorized data.
- **Usability:** The application must be web-based but fully responsive, as the primary end-users (students) will access the platform via mobile phones.
- **Reliability:** Database transactions must be ACID-compliant to prevent data corruption during simultaneous user enrollments or assignment creations.

2.2 Design Phase

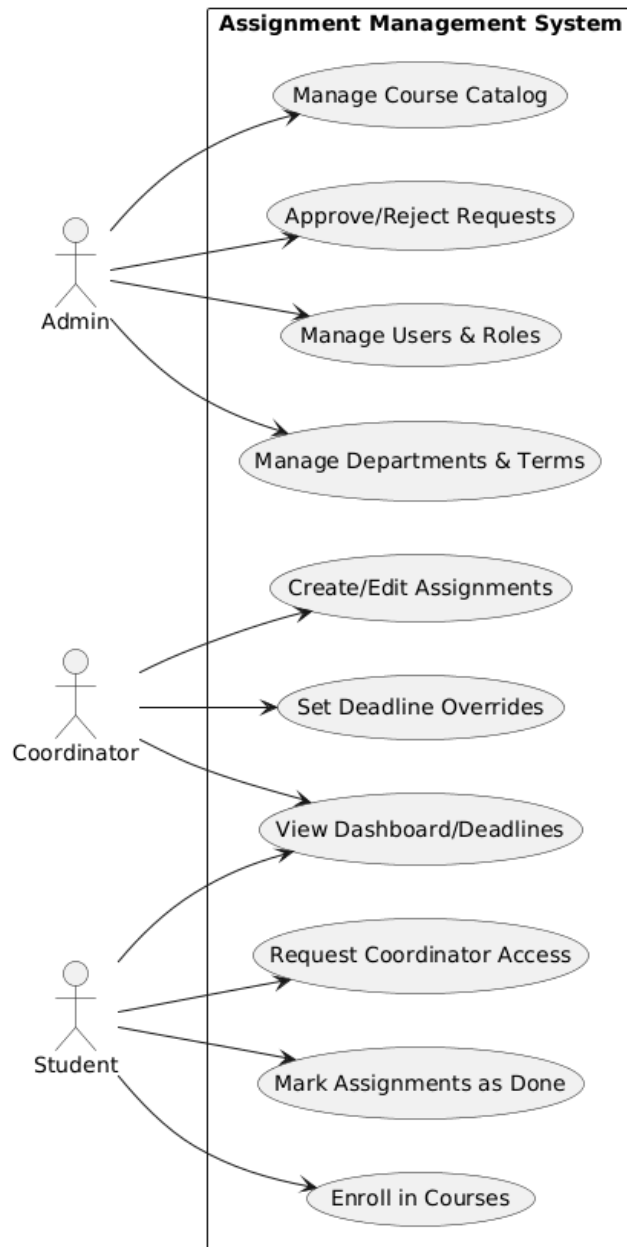
System Architecture: The system is built on a monolithic Next.js architecture utilizing the App Router, combining both the frontend UI and the backend API routes into a single repository. The backend communicates directly with a PostgreSQL database using the lightweight pg driver (node-postgres), executing raw SQL queries rather than relying on an ORM abstraction to maintain strict control over complex table joins.

Key Design Decisions:

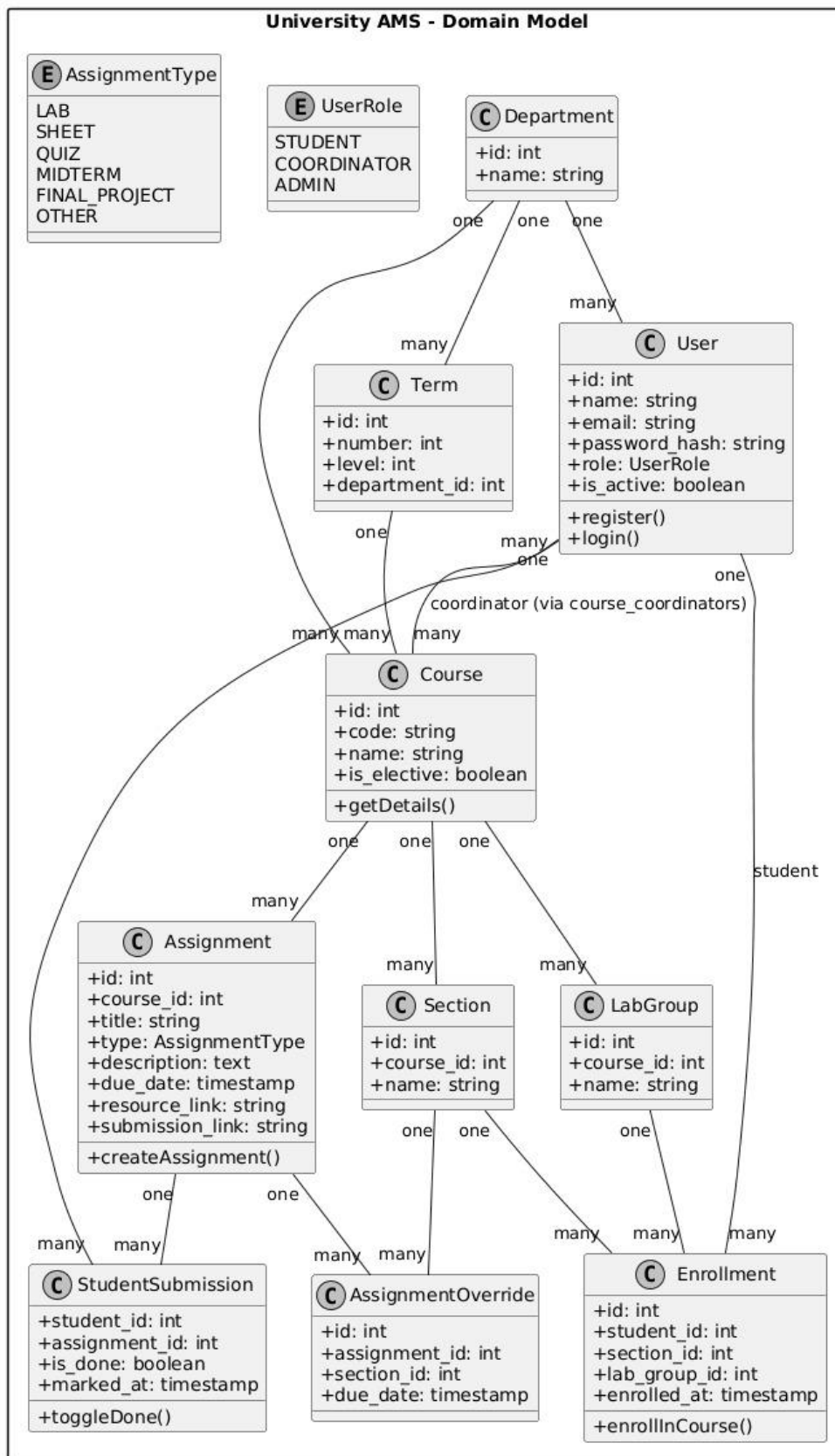
1. **Course Identity:** A course code is treated as unique per department and term. Identical course names in different departments are modelled as entirely separate courses to eliminate ambiguity.
2. **Deadline Model (The Override Pattern):** To prevent duplicate records, the database schema utilizes a single assignments table containing a default due date. A secondary assignment_overrides table stores exceptions linked to specific section IDs.
3. **Authentication:** NextAuth.js was chosen to handle JWT sessions securely out of the box.

UML Diagrams:

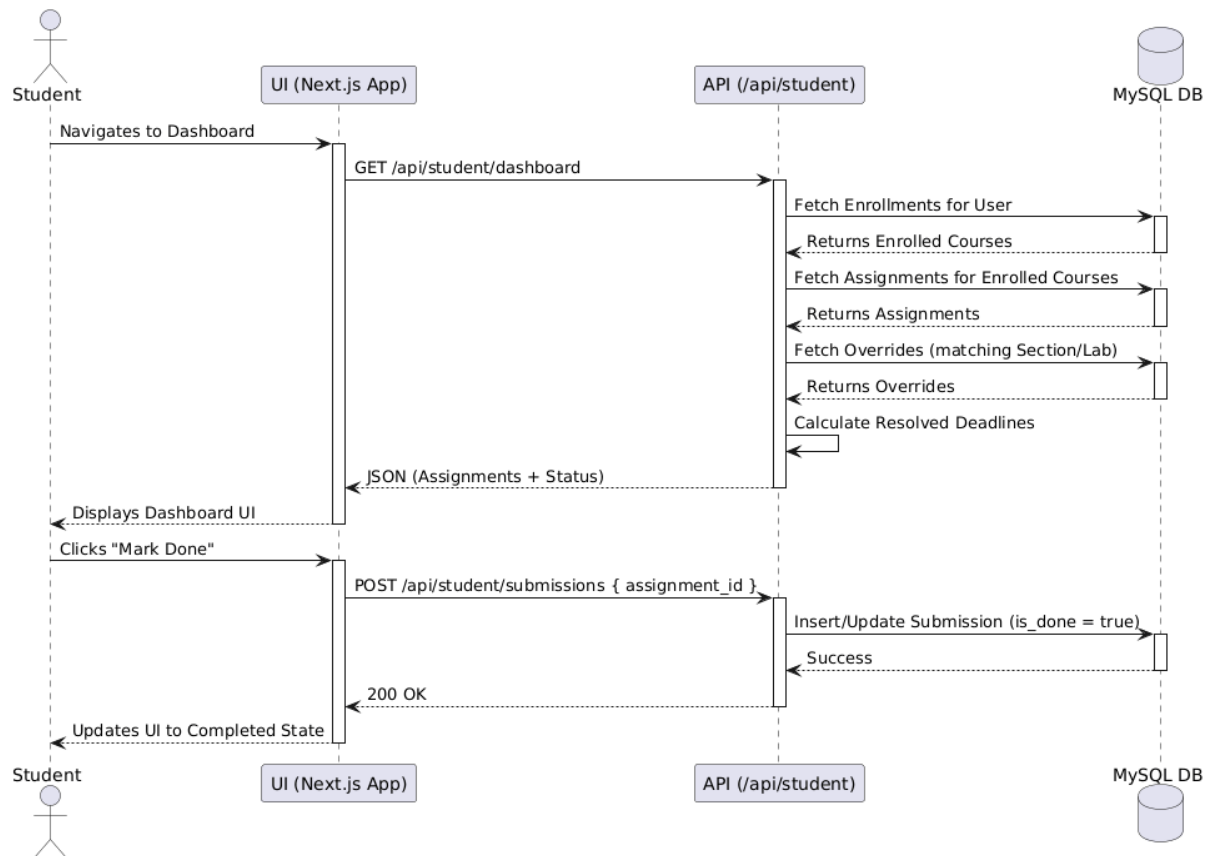
1. **Use Case Diagram:** Illustrates the boundaries of the system and the distinct capabilities of the Student, Coordinator, and Admin actors.



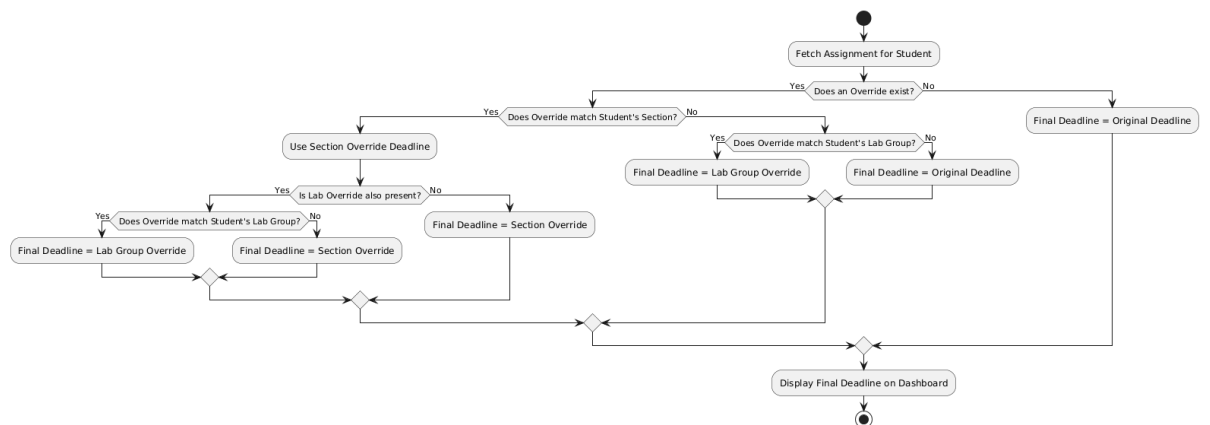
2. Class Diagram:



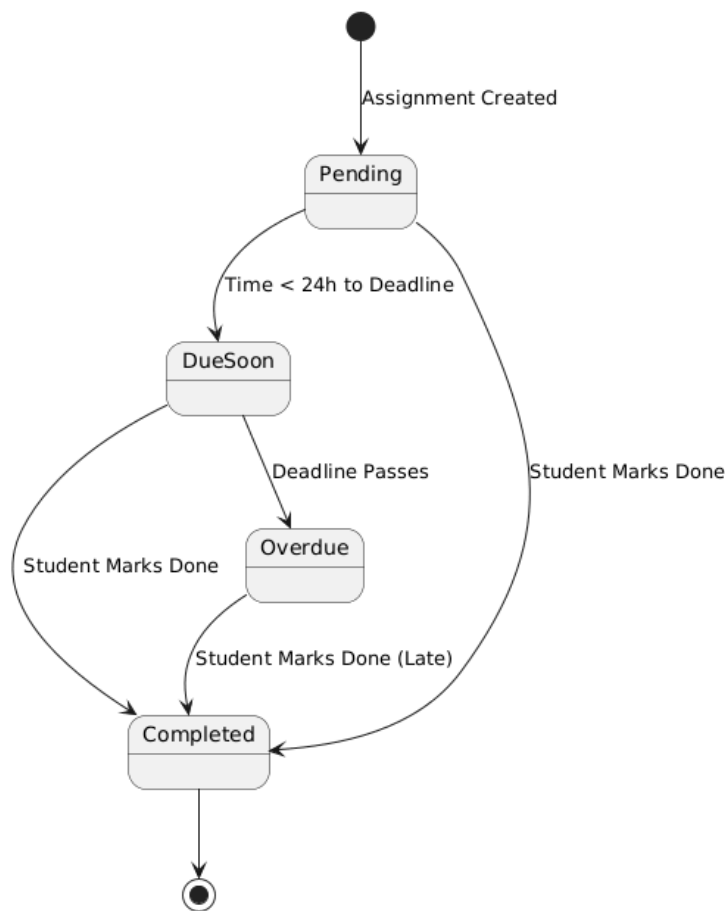
3. Sequence Diagram:



4. Activity Diagram:



5. State Machine Diagram:



2.3 Implementation Phase

Technologies and Tools Used:

- **Framework:** Next.js (App Router)
- **Language:** TypeScript for type safety and shared data models across the team.
- **Database:** PostgreSQL, accessed via the pg driver.
- **Authentication:** NextAuth.js.
- **Styling & UI:** Tailwind CSS combined with pre-built accessible components from shadcn/ui.
- **Containerization:** Docker & Docker Compose for localized, one-command environment setups.

Development Process: The software was implemented using an iterative Git workflow. The team utilized a dev integration branch, and developers pushed features via

individual branches (e.g., feature/assignment-overrides). Pull requests were reviewed and merged daily, and progress was tracked using 15-minute daily stand-ups and a shared GitHub Project board.

Key Modules:

- **API Route Handlers:** Located in `app/api/`, acting as the backend to process CRUD operations for departments, courses, and assignments.
- **Database Config:** The `lib/db.js` module handles the PostgreSQL connection pooling.
- **Role-Based Layouts:** The application logic is modularized into `app/(student)/`, `app/(coordinator)/`, and `app/(admin)/` route groups to strictly isolate role-specific interfaces.

2.4 Testing Phase

Testing Strategies: The system's correctness was verified using a combination of automated backend testing and manual end-to-end integration testing. Postman was utilized heavily to verify the API routes prior to connecting the frontend.

Test Cases:

- **Unit Testing (Deadline Resolution):** Given an assignment with a default deadline of Friday, and an override for "Section 1" set to Monday. Verified that a student in Section 2 receives the Friday deadline, while a student in Section 1 receives the Monday deadline.
- **Integration Testing (Coordinator Flow):** Verified that a newly registered Coordinator account lands in a "pending approval" state, and that the Admin dashboard successfully fetches and approves the user, updating their database role and unlocking the coordinator dashboard.
- **System Testing (End-to-End):** Verified the complete student flow: signing up, browsing the course catalogue, self-enrolling in a lab group, and marking an overdue assignment as "done" to trigger the correct UI state changes.

2.5 Maintenance Phase

System Maintenance: The use of TypeScript ensures that future engineers can maintain the system with minimal bugs by providing strong typing for database structures and API payloads. The database schema is fully relational and normalized, preventing data anomalies and making future schema migrations straightforward.

Future Improvements:

While the current MVP successfully handles core tracking, planned future upgrades include:

- Implementing email notifications using Nodemailer and a Gmail SMTP to alert students when a deadline is under 24 hours away.
- Building an analytics dashboard for Coordinators to view real-time counts of how many students have marked an assignment as completed.
- Adding automated CSV exports for assignment lists.
- Creating a comprehensive audit log tracking which users created, edited, or deleted specific records.

Scalability: The Next.js architecture allows the application to be deployed and scaled rapidly using serverless environments. Relying strictly on raw SQL queries via the pg driver eliminates ORM bloat, guaranteeing that database calls remain highly performant even as the user base expands across the university.